

Assignment #3

Points: 8

Deadline for submission: 17 April 2026, 23:59

1. Objective

In **Assignment 1**, you designed the **microservices architecture** of your system.

In **Assignment 2**, you developed the **domain model** describing the main domain concepts and relationships.

In this assignment, you will refine your system design to prepare it for the **implementation phase of the project**.

The goal of this assignment is to:

- define the **REST APIs** of the services
- define the **data model** of each service
- describe **how services collaborate through workflows**
- demonstrate **service integration** using both synchronous and asynchronous communication
- define the **final implementation responsibilities** of each team member

The design produced in this assignment will serve as the **implementation plan for the project**.

2. Deliverable

Each team must submit **one document** containing the following sections:

- Complete system architecture (all services)
- Service APIs
- Service data models
- System workflows
- Integration mechanisms
- Final implementation responsibilities

The document must clearly indicate **which student is responsible for each component to implement**.

3. Complete System Architecture

Provide the **complete architecture of the system**, including all microservices defined in **Assignment 1**.

This architecture must include:

- all services
- their main responsibilities
- the relationships between services

Even if some services will not be implemented in the project, they must still be **represented in the system architecture** to preserve the overall business logic of the system.

After presenting the complete architecture, clearly specify the **subset of services and components that will be implemented during the project** (see Section 7).

In addition to the explanation, provide a diagram showing the services and their relationships.

4. Service APIs

Each student must define the **REST API endpoints** for the services they are responsible for implementing.

For each service, provide a table describing the endpoints. For example:

Endpoint	Method	Description
/bookings	POST	Create a booking
/bookings/{id}	GET	Retrieve booking details
/bookings/{id}	DELETE	Cancel a booking

Services in this project are expected to contain a **relatively small number of endpoints**, typically around **5–8 endpoints per service**.

The APIs must be consistent with:

- the **system functionalities defined in Assignment 1**
- the **domain model defined in Assignment 2**

5. Service Data Models

Each student must define the **data model for the services they will implement**.

Provide a simple entity relationship **ER-style diagram or table description** showing:

- entities
- attributes
- relationships between entities

Each microservice must manage **its own database**. Microservices must **not share databases directly**.

6. System Workflows

Describe the **main workflows of your system**, showing how multiple microservices collaborate to implement the system functionalities defined in your previous assignments.

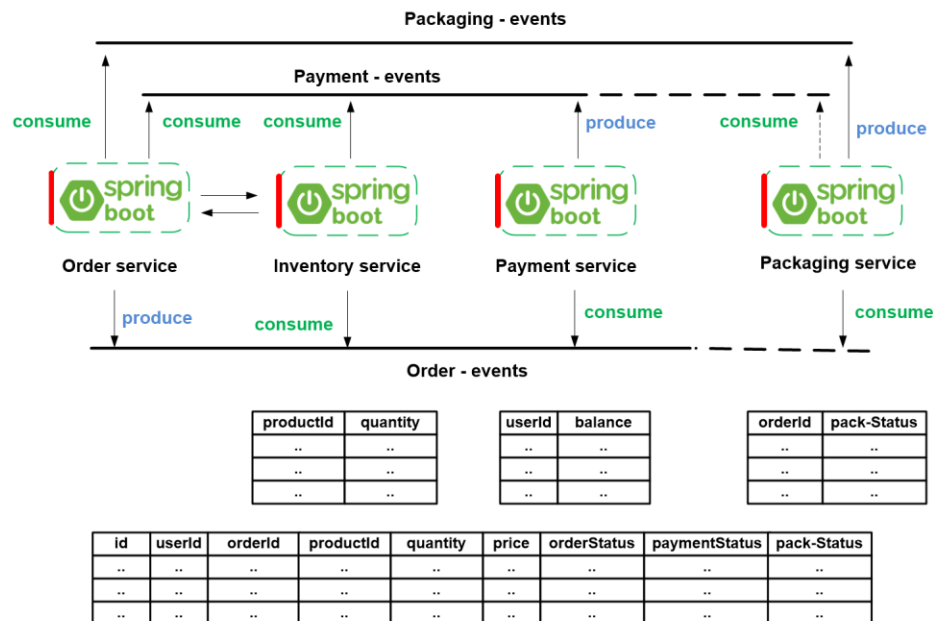
A workflow represents the sequence of interactions between services that occurs when a user performs an operation in the system (for example: creating an order, booking a trip, confirming a payment, etc.).

Each workflow diagram should indicate:

- the microservices involved in the workflow
- the interactions between services

- the events produced and/or consumed by services (if applicable)
- the channels or topics used to communicate such events
- the producer and consumer(s) of each event
- the data tables influenced or updated by these interactions

For example, the following figure shows a **partial workflow model** in an event-driven architecture:



The workflows **selected for implementation in the project** must include both interaction styles:

- **at least one synchronous interaction between services (e.g., REST calls)**
- **at least one asynchronous interaction using events or messaging.**

7. Implementation Responsibilities

Each student is initially responsible for implementing **two microservices (from your Assignment 1)**.

However, to support system integration, each student may replace **one of these services by a shared integration or resilience component**, such as:

- API Gateway
- Service Discovery
- Load Balancing configuration
- Messaging/event infrastructure
- Resilience mechanism
- other integration or resilience component discussed in the course or justified by your system design.

If you replace one service and implement a shared component:

- the component must be **used by the system**
- the responsible student must be **clearly identified**
- the component must contribute to **system integration/resilience**

Important: Each student may replace **at most one microservice with an integration/resilience component**.

If you replace one of the two services with a shared component, the remaining business service must:

- expose a **non-trivial REST API** (typically several endpoints)
- manage **its own persistent data**
- participate in **at least two interactions with other services**
- participate in **at least one of the implemented system workflows**

Each team must clearly specify:

- which services will be **implemented**
- which services remain **only part of the design**
- which services are **replaced by shared integration components**
- which workflows will be **implemented in the project**

Please, be aware that the final system must remain **realistic and coherent**, with enough business services to implement the main functionalities of the system.

8. Submission Rules

- Submit **one PDF or Word file per team** through Moodle.
- The document must include **the names of all team members**.
- Only **one student per team needs to submit the assignment**.
- Late submissions will lose **0.5 points per hour**.

9. Final Remarks

The design produced in this assignment will be **implemented during the project phase**.

Keep in mind that:

- the goal is to demonstrate **service interaction and system integration**
- the system architecture and implementation should remain simple, realistic, and feasible to implement within the project timeline.
- the project should focus on **core functionality rather than complex infrastructure**